

COMS W4156 Advanced Software Engineering (ASE)

September 28, 2023



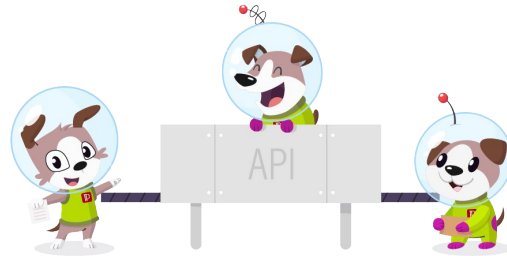
Does Every Team Know Who Is Their Mentor?

https://docs.google.com/spreadsheets/d/1P6ZQZaTucTLPEnlfcU73xXfmXj0M1HrAm_hcVQ7Mu_g/edit#gid=0



Agenda

1. **API Documentation**
2. API Testing
3. if time permits: Equivalence Partitions and Boundary Analysis



Why Document Your API?

If you're building an API for an open-source or proprietary third-party library or framework, or a free-to-use or commercial service, you want someone who is not you to use it!

Your API is the product and your API documentation is the manual, It doesn't matter how good your API is if potential adopters don't understand how to use it

Your API's documentation is the primary resource for explaining what is possible with your API and how to get started

It also serves as a place for developers to return with questions about syntax or functionality

The best API docs answer these questions (so you don't have to)



What Is Included In API Documentation

Tells other developers what your API can do for them and why they would want to use it - what are the [use cases](#)?

May compare to alternatives, e.g., yours is better, faster, cheaper

Tells other developers how to use your API - what to do and what not to do

Covers every entry point's parameters and return values, covers authentication if applicable (are you allowed to do that?)

Lists all possible status or error codes it might return, explaining what they mean, what could cause this to occur, and how to handle it

Includes realistic code examples, preferably at least one for every entry point

Not intended for end-users



Examples of Good API Documentation

Let's look at them to see what is Good

[Google Maps API documentation](#)

[Slack API documentation](#)

[Microsoft technical documentation](#)

Where I got these Good examples from, also lists some others:

<https://blog.dreamfactory.com/8-api-documentation-examples/>



Bad API Documentation: Can't Find It or It's Not There

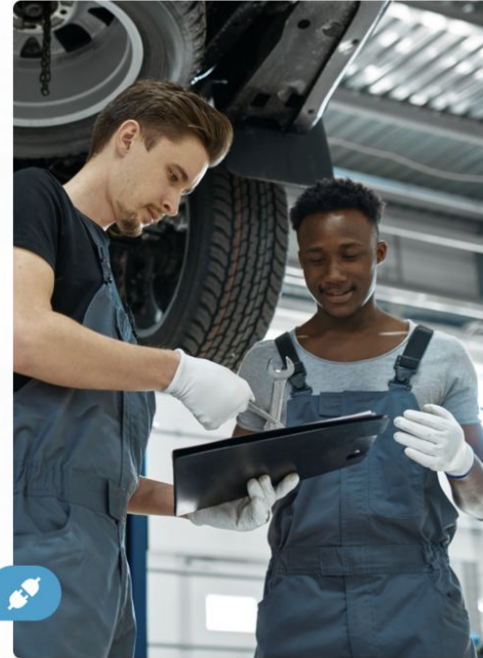
How does our API work?

1. A client application initiates an API call to retrieve information also known as a request. This request is processed from an application to the web server via the API's Uniform Resource Identifier (URI) and includes a request verb, headers, and sometimes, a request body.
2. After receiving a valid request, the API makes a call to the external program or web server.
3. The server sends a response to the API with the requested information.
4. The API transfers the data to the initial requesting application.

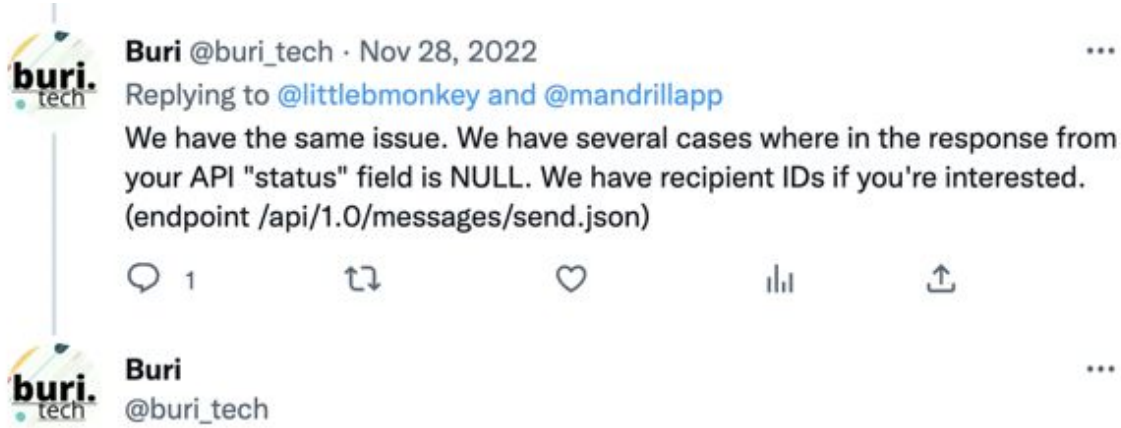
The data transfer will vary depending on the web service being used. Regardless, this process of requests and response all happens through an API.

A user interface is designed for use by humans; APIs are designed for use by a computer or application.

API documentation available upon request.



Bad API Documentation: Broken, Inaccurate, and Outdated



I found out what's wrong! When email sending is soft-bounced or hard-bounced then "status" field is null! This is not documented in API documentation. Is it a bug or a feature? :)

Bad API Documentation: Click Here to Download the Docs



 **USPS.COM**

Quick Tools

Send

API User Guides

Address APIs - [PDF](#) | [HTM](#)

- Address Validation/Standardization
- City & State Lookup
- Zip Code™ Lookup

Tracking & Delivery Information APIs - [PDF](#) | [HTM](#)

- Tracking
- Tracking by Email

Bad API Documentation: The One-Trick Pony

 **swagger**

Explore

Swagger Petstore ^{1.0.0}

[Base URL: petstore.swagger.io/v2]
<https://petstore.swagger.io/v2/swagger.json>

This is a sample server Petstore server. You can find out more about Swagger at <http://swagger.io> or on [#swagger](irc://freenode.net). For this sample, you can use the api key **special-key** to test the authorization filters.

[Terms of service](#)
[Contact the developer](#)
[Apache 2.0](#)
[Find out more about Swagger](#)

Schemes

HTTPS ▾

Authorize 

pet Everything about your Pets

Find out more: <http://swagger.io> ▾

POST

/pet Add a new pet to the store



PUT

/pet Update an existing pet



GET

/pet/findByStatus Finds Pets by status



Where I Got The Bad Examples

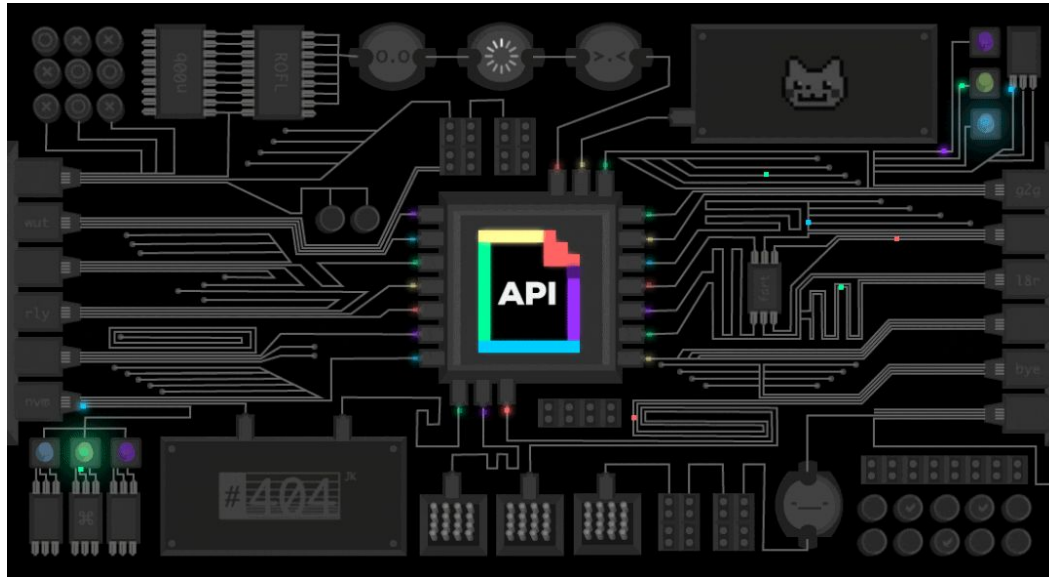
<https://everydeveloper.com/bad-documentation-examples/>

Examines what makes them Bad

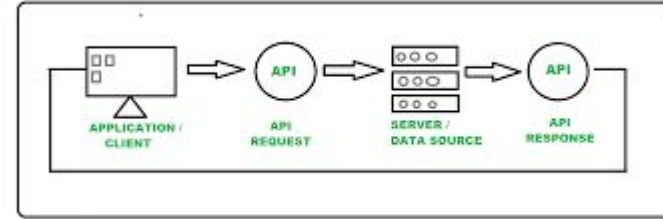


Agenda

1. API Documentation
2. API Testing
3. if time permits: Equivalence Partitions and Boundary Analysis



API Testing



API Testing needs to cover:

- Synchronous call/response and asynchronous events (as applicable),
- Errors visible at API interface, and
- API functions that read/write the internals of shared data or affect shared resources

Error messages (or lack thereof) are typically the last thing developers think about but the first thing users notice when software does not work as users expected

This includes developers who are users of APIs. But error messages need to be appropriate for code to code not display to humans - e.g., status codes not strings (but documentation needs to explain each status code with human-readable text)

For Each API Entry point (aka Endpoint)

Provide inputs intended to force 'happy path' (or paths) to make sure basic functionality works. [Happy Path](#) = typical valid input where (most likely) nothing can go wrong

Then try to call the entry point without a mandatory parameter, a parameter of the wrong type, and parameters out of order



Devise inputs to try to force every status code that the entry point could return. If the caller is supposed to “do something” for a status code that affects the API, test with and without actually doing it (do something else instead)

If the caller needs to supply callback code for API events, test with mock callers that do and do not actually fulfill the expected interface, and do / do not actually perform any expected actions that affect the API

For Each Inter-related Collection of API Entrypoints

Call the API entry points in the expected sequence

Call the API entry points in various orders that are unexpected, e.g., calling x sets up state and calling y uses that state, so call y without calling x first and call x multiple times in a row

Make combinations of API calls designed to trigger errors

Also test any API entry points used by your own code!
Make sure they do what you think they do...



Consider Where API Calls Might Come From



Some API Testing Tools

- [Postman](#)
- [SOAPUI](#)
- [cURL](#)
- [Apache JMeter](#)
- [gRPC testing](#)
- [“10 Best API Testing Tools in 2023”](#)
- [“10 API security testing tools to mitigate risk”](#)



Security testing will be discussed later in semester

Agenda

1. API Documentation
2. API Testing
3. if time permits: Equivalence Partitions and Boundary Analysis

<i>Testing</i>	
Advantages	Disadvantages
Provide a sense of complete testing and eradicates the need for exhaustive testing	Suitable only for range-wise and discrete values input data
Enables large domain of inputs or outputs coverage with a smaller subset selected from an equivalence class	Assumes that the data in the same equivalence class is processed in the same way by the system

Baseline Testing Strategy

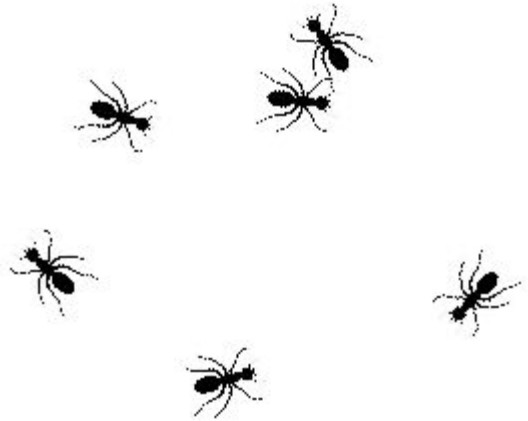
- *Test with one typical valid input, one atypical valid input, and one invalid input
[better: one invalid value right format, one invalid wrong format]*

It's better than no strategy

It's better than the common CS1 strategy -
test program with two or three typical valid inputs,
done

It applies to unit testing, integration testing,
system testing, API testing, ...

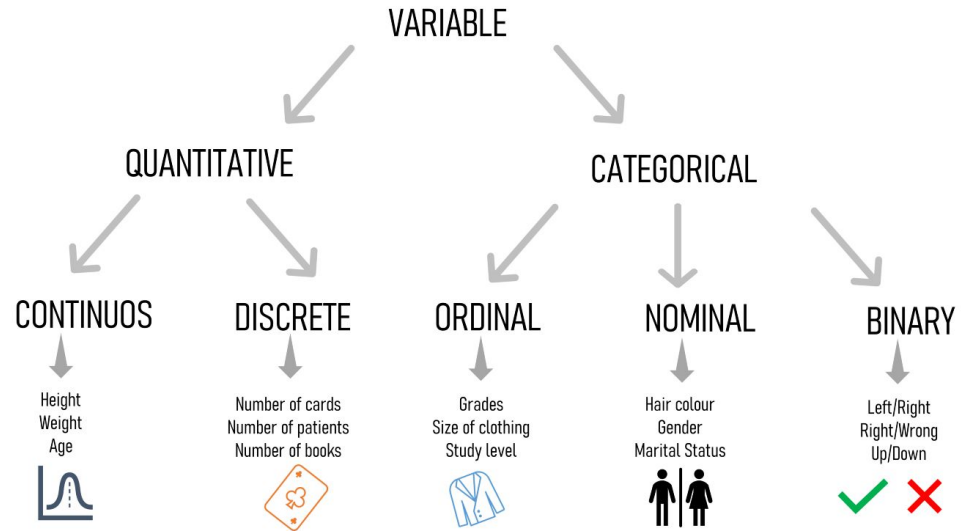
Think of it as just a starting point



Better Approaches to Choosing Test Inputs

If you have an automated coverage tool, or your program is small enough to check manually, try to achieve 100% coverage

Also test with a set of inputs that you think would make a nice demo - but keep in mind what might happen when the demo audience asks to “drive”. If this is higher level management in your company, your venture capital investor, or the instructor/TA who decides your grade, you cannot say No



How About Testing ALL Possible Inputs?

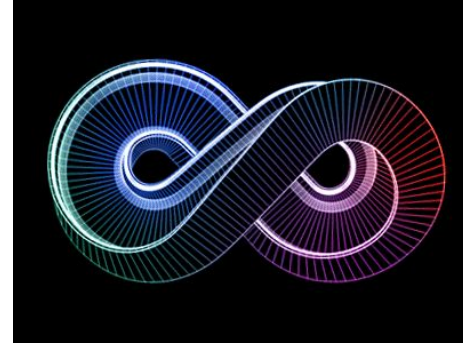
“Exhaustive Testing”

Usually “infinite” number of valid and invalid inputs
(assuming infinite size input buffers available)

Even if number of valid and invalid inputs is finite,
infeasible to try all of them

To maximize the chances of finding and fixing
bugs, we need to choose a practically small
number of inputs that, collectively,

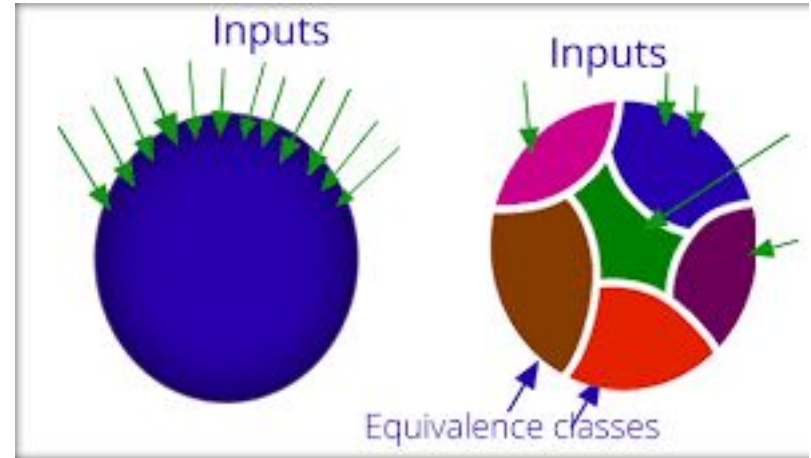
- represent all possible inputs and
- are *most likely* to reveal bugs



Equivalence Partitions

The simplest approach for representing all possible inputs to a system with a practically small number of examples is to divide the input space into *equivalence partitions* (also known as equivalence classes) based on **both**

- domain knowledge
- software engineering knowledge (where do bugs like to hide?)



Equivalence Partitions

Equivalence partition = set of alternative input values where the software can reasonably be expected to behave equivalently (similarly)

This does **not** mean literally the same output for every input in the class!



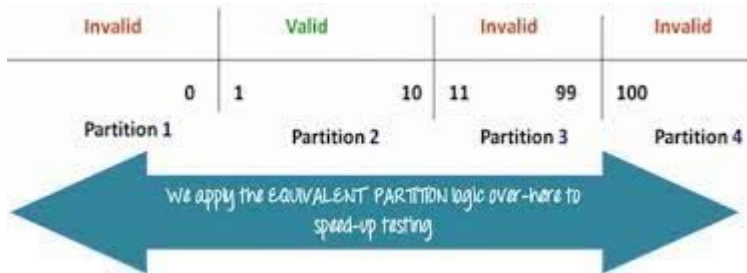
Instead it means that if the software behaves correctly on one example input chosen from that equivalence class, it is reasonable to believe it will behave correctly on all the other inputs in the same class

Think about what kind of computation or processing is needed for different members of the class - if it's not very similar then it's not actually an equivalence class and you need to rethink your partitioning strategy

Simple Example

If we are testing a “box” intended to accept numbers from 1 to 10, then do not write ten test cases for all 10 valid input numbers

If we are testing a “box” intended to accept numbers from 1 to 1000000, then do not write a million test cases for all 1000000 valid input numbers!



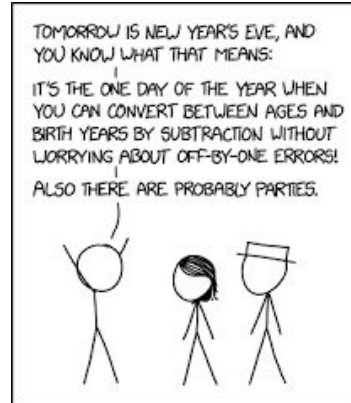
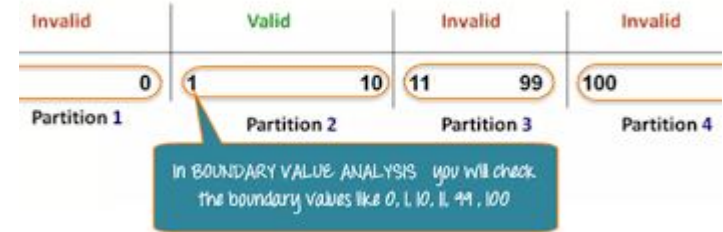
Instead, divide the test cases into three sets:

- 1) Equivalence partition with all valid inputs. Pick a single value from range 1 to 1000000 as a valid test case. If we select other values between 1 and 1000000 the result “should” also be correct (or also be incorrect)
- 2) Pick a single value for the partition with all values *below* the lower limit of the range, i.e., any value below 1
- 3) Pick a single value for the partition with all values *above* the upper limit, i.e., any value above 1000000

➤ Are we done yet?

Boundary Value Analysis (or just Boundary Analysis)

Equivalence partitions are typically extended with boundary analysis because input values at the extreme ends of the valid and invalid partitions are more likely to trigger bugs = “off by one error”



Boundary Value Analysis

- 1) Test cases with input exactly at the boundaries of the valid partition, i.e., values 1 and 1000000
- 2) Test data with values just below the extreme edges, i.e., values 0 and 999999
- 3) Test data with values just above the extreme edges, i.e., values 2 and 1000001

For integer domains, make sure to include zero, positive and negative examples, as well as the underlying primitive extremes

In this case, we already have 0 and some positive values, so we need to add some negative value, e.g., -1, plus MinInt and MaxInt

OFF-BY-ONE ERROR EXAMPLES

The loop should run n times. n can be any integer value.

```
for (int i = 1; i < n; i++) {  
    /* Body of the loop */  
}
```

i	n	loop
1	5	runs
2	5	runs
3	5	runs
4	5	runs
5	5	no run

<http://www.paulhogan.com/>

stopped here

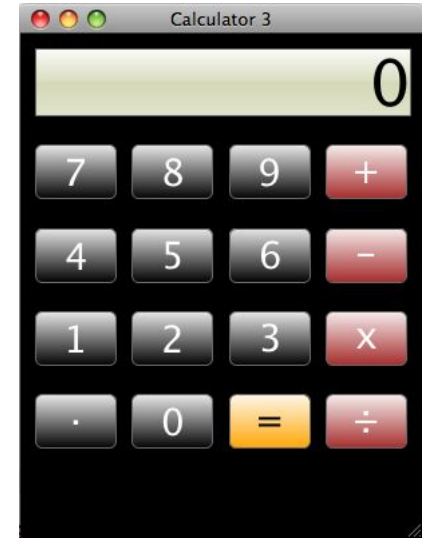
Another Example

Consider a calculator implemented in software, with a GUI as shown. The user clicks the buttons with a mouse, but cannot enter text into the display

In mathematics, the “+” operation takes two numeric operands

Say this software is tested by an external testing team separate from the developers

Based on *domain knowledge*, a tester might reasonably expect the “+” operation to behave similarly on all pairs of numbers
< first operand, second operand >

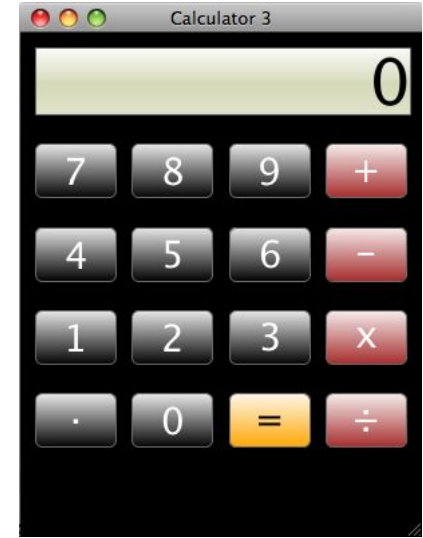


Another Example

So the tester tries an arbitrary pair of numbers:

< 3.141592653, 42 >

The calculator flashes 00000000. What does this mean?



Interpreting Test Outputs

Based on *software engineering knowledge*, the tester might realize the flashing 00000000 could indicate:

- An error message because the software cannot handle numbers with more than N digits (e.g., 8)
- An error message because the software cannot handle mixed operands, i.e., one operand is floating point and the other is integer
- A bug
- Something else (e.g., the user has reached the limit of their “free trial” and now has to pay to continue using the calculator software)



Continuing Calculator Example

Let's ignore the “.” key (planned for version 2.0) and assume the “+” operator is currently intended only for integer operands

A tester with software engineering knowledge, not just domain knowledge, might realize that a calculator implementation could behave differently for positive integers, zero, and negative integers

Thus they test with five different equivalence classes:

- < pos int, pos int >
- < pos int, neg int >
- < pos int, 0 >
- < neg int, neg int >
- < 0, 0 >

Is this sufficient?



Software is not Mathematics

Although the “+” operator is mathematically commutative, that does not mean that the implementation (which could be buggy) is commutative

Or, more generally, define equivalence partitions separately per parameter, and then extend to the cross-product

But notice the scaling problem with large numbers of parameters!

- $\langle \text{pos int}, \text{pos int} \rangle$
- $\langle \text{pos int}, \text{neg int} \rangle$
- $\langle \text{pos int}, 0 \rangle$
- $\langle \text{neg int}, \text{pos int} \rangle$
- $\langle \text{neg int}, \text{neg int} \rangle$
- $\langle \text{neg int}, 0 \rangle$
- $\langle 0, \text{pos int} \rangle$
- $\langle 0, \text{neg int} \rangle$
- $\langle 0, 0 \rangle$



Extremes

Let's say the calculator is intended to handle operands only up to 8 digits (or to any other fixed number of digits)

This means there is a `MaxInt` and a `MinInt` (negative), probably specific to what the calculator GUI was designed to display rather than `MaxInt/MinInt` of the programming language, compiler, operating system, etc.

What additional tests are needed?

- `< xxx, MaxInt >`
- `< MaxInt, xxx >`
- `< xxx, MinInt >`
- `< MinInt, xxx >`
- `< MaxInt, MinInt >`
- `< MinInt, MaxInt >`



Invalid Inputs to Calculator Example



What if the user could enter text into the display window in addition to mousing the keys

How does that change the set of equivalence partitions that need to be tested?

Another Example

What are the
equivalence partitions?

How would we test this
program?

Design-your-burger

Please make your selection

Burger:

Cooked: ☐ rare ☐ medium ☐ well

Cheese: ☐

Lettuce: ☐

Tomato: ☐

Onion: ☐

Ketchup: ☐

Mustard: ☐

Mayo: ☐

Secret sauce: ☐



[Nutrition facts](#) [About the chef](#)

Required vs Optional Inputs

Get

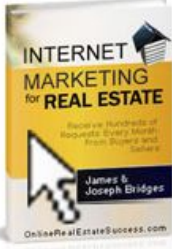
"Internet Marketing for Real Estate EBook"

via e-mail

Name*:

Email*:

Phone*:



* Indicates a required field

* Indicates an optional field

Equivalence classes need to address required vs. optional inputs, with or without default values

What are the equivalence partitions?

How would we test this program?

Dependencies between Inputs

Distinct parameters may be related to each other in forming equivalence classes

Let's say we had a function that took two variables (blood sugar level and level of frustration) to calculate "mood"



Assume valid inputs are
Blood sugar 70 - 100
Frustration is 50% - 100%

For $50 \leq F \leq 90$

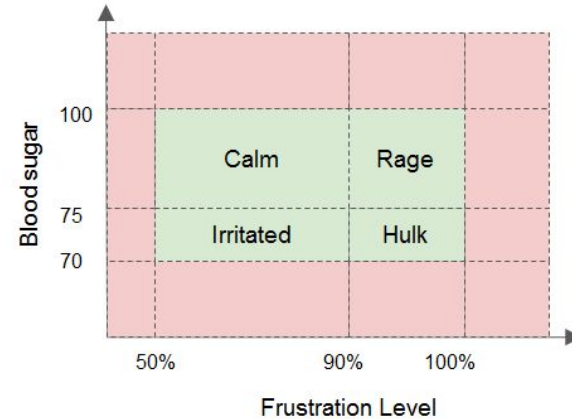
75 $\leq$ BS $\leq$ 100: Calm

BS < 75: Irritated

For $F > 90$

75 $\leq$ BS $\leq$ 100: Rage

BS < 75: [Hulk Smash](#)



More Dependencies between Inputs

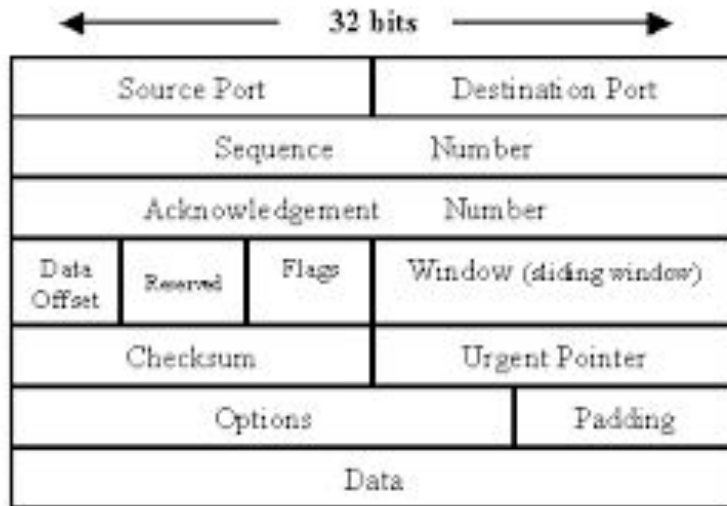
Equivalence classes may cut across multiple inputs:



Sometimes the value of one input restricts appropriate values for other inputs. For example, date of birth and age should be consistent (valid) or not (invalid)

Sometimes the value of one (small) input is intended as a sanity check on another (larger, more complicated) input, such as the checksum in the network packet

System to System Inputs (includes APIs)



JSON Object → {

String Value ↓

Object Inside Object → {

JSON Array → [

Array Inside Array → [

Number Value → 102

Null Value → null

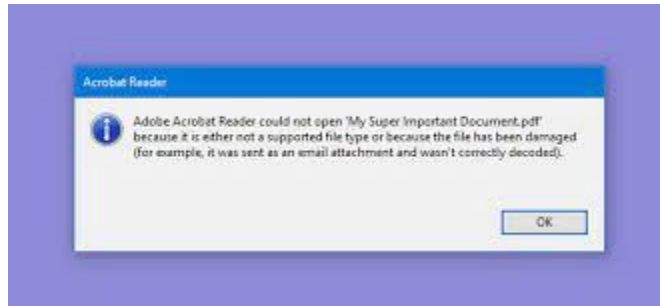
```
{
  "company": "mycompany",
  "companycontacts": {
    "phone": "123-123-1234",
    "email": "myemail@domain.com"
  },
  "employees": [
    {
      "id": 101,
      "name": "John",
      "contacts": [
        "email1@employee1.com",
        "email2@employee1.com"
      ]
    },
    {
      "id": 102,
      "name": "William",
      "contacts": null
    }
  ]
}
```


System and Service Inputs

At the full system and service levels, inputs are generally going to be numbers, strings (text), tuples (packets, json), and files

File inputs can:

- Exist or not
- Be below min or above max size
- Be readable, writeable, executable by current user (permissions)
- Correct, corrupted, garbled, etc. data and metadata formats - images, audio, video, ...



Unit Inputs

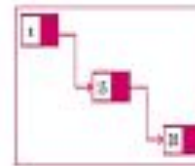
At the unit testing level, inputs/outputs can be arbitrary data structures, so equivalence classes can be more complex - and inputs more complicated to construct

Container data structures with content elements:

- In or not in container
- Empty or full container (or min/max number of elements)
- Duplicate elements may or may not be allowed
- Ordered or organized correctly, or not



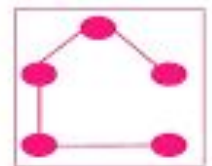
Sorting



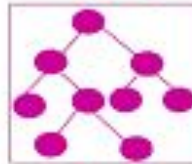
Link list



list



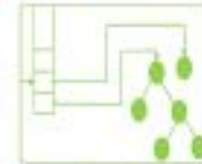
spanning tree



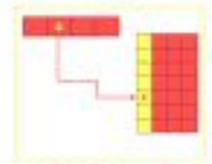
Tree



Graph

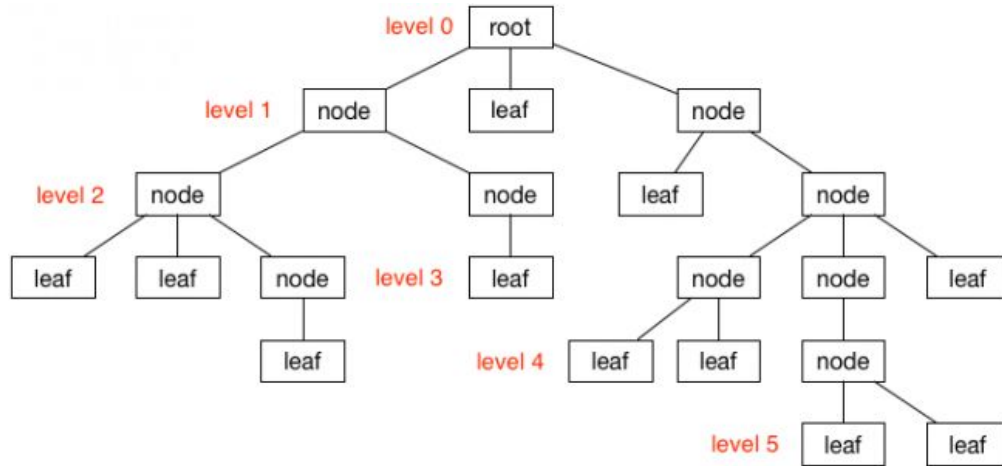


Stack



Hashing

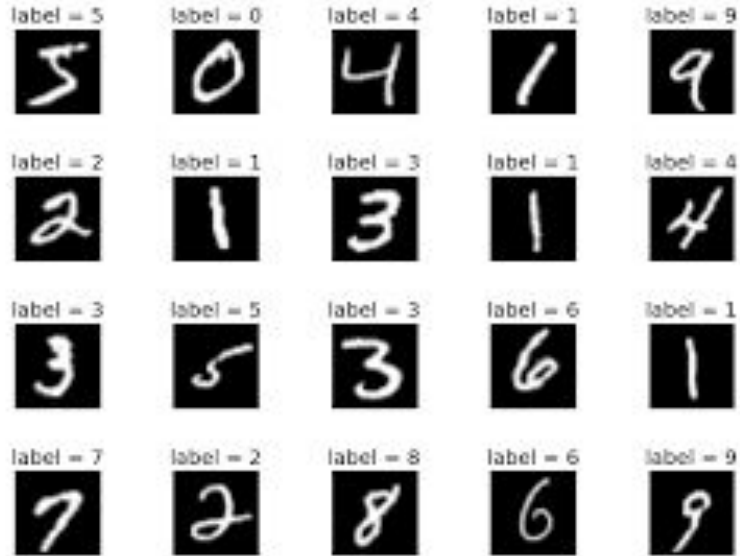
More Unit Inputs



Specific kinds of containers, e.g., tree - root node, interior node, leaf node, null tree, tree with exactly one node, “full” tree, balanced vs. unbalanced, various specialty trees - and graphs

How would test cases construct root, node and leaf objects to use as inputs?

Equivalence Partitions for Data Sets



Files containing “data sets” often correspond to a table (rows and columns)

This shows part of a well-known dataset for deep learning algorithms ([MNIST](#))

What are the equivalence classes?

Upcoming Assignments: Note updated deadlines!

- [Team Revised Project Proposal](#) due next Thursday, October 5, by 11:59pm
- [First Iteration](#) due Thursday, October 19, by 11:59pm
- [First Iteration Demo](#) due Thursday, October 26, by 11:59pm

Get Going ↗

Next Week

If needed: Finish Equivalence Partitions/Boundary Analysis

Unit Testing



Ask Me Anything